

Mechatronics and Industrial Internet of Things

IIoT-Based Smart Refrigerator Monitoring System

ASSIGNMENT - 8

TEAM MEMBERS

- **JAGADEEP S – 24MER018**
- **JEEVA P – 24MER019**
- **JOGINDER S – 24MER021**
- **RAGUL R – 24MER046**

1. PROJECT OVERVIEW

The Smart Refrigerator Monitoring System is an IoT-based solution developed to continuously monitor and maintain suitable environmental conditions inside a refrigerator. Temperature and humidity are important factors that directly influence the freshness, quality, and storage life of food products. Improper temperature levels or excessive humidity can accelerate food spoilage, promote microbial growth, and affect the performance of the refrigeration system.

Traditional refrigerator monitoring mainly depends on manually checking the temperature or observing the refrigerator display. Such methods do not provide continuous data collection and may not immediately identify sudden temperature increases caused by frequent door opening, power interruptions, cooling-system problems, or sensor abnormalities. Therefore, an automated monitoring system is required to provide continuous observation and timely information about changing refrigerator conditions.

The proposed system uses a temperature and humidity sensor to periodically measure the internal environmental conditions. The sensor readings are collected by a microcontroller and processed before being transmitted through a Wi-Fi connection. The collected information is uploaded to a cloud-based monitoring platform, allowing users to access the refrigerator data remotely through an internet-connected device.

The system can continuously record temperature and humidity values at predefined time intervals, creating a historical record of refrigerator conditions. This historical data can be used to identify temperature fluctuations, unusual humidity changes, and long-term operating patterns. When the measured values exceed predefined safe limits, the system can generate an alert to indicate that the refrigerator may require attention.

The system also helps reduce the possibility of unnoticed food-storage problems by providing real-time monitoring rather than relying only on manual inspection. Remote monitoring is particularly useful when the refrigerator is located away from the user or when continuous observation is required. The collected data can also support preventive maintenance by identifying abnormal environmental conditions that may indicate cooling or operational issues. The Smart Refrigerator Monitoring System demonstrates the practical application of IoT technology in smart homes, food-storage facilities, laboratories, restaurants, supermarkets, and other environments where temperature-sensitive materials need to be monitored. The system can be further improved by integrating mobile notifications, cloud-based data analysis, automatic alarm systems, predictive maintenance, and energy-consumption monitoring.

Overall, the proposed system provides a cost-effective and automated approach for continuous refrigerator monitoring. By combining sensor-based measurement, Wi-Fi communication, cloud storage, remote access, and automatic alerts, the system improves environmental monitoring, supports food safety, and provides useful information for maintaining efficient refrigerator operation.

2. PROJECT STATEMENT

Food stored in refrigerators requires suitable temperature and humidity conditions for proper preservation. Manual checking of refrigerator conditions does not provide continuous monitoring and may fail to detect sudden changes. Such variations can affect food quality, increase the risk of spoilage, and reduce the overall efficiency of the refrigeration system.

Therefore, an IoT-based Smart Refrigerator Monitoring System is proposed to continuously monitor the internal environmental conditions of a refrigerator and provide remote access to the collected data through a cloud platform. The system is designed to:

- Continuously measure refrigerator temperature.
- Continuously measure refrigerator humidity.
- Monitor environmental conditions in real time.
- Detect abnormal temperature and humidity variations.
- Transmit sensor data through Wi-Fi.
- Store sensor readings on a cloud platform.
- Enable remote monitoring through a dashboard.
- Support historical data analysis for performance evaluation.

The proposed system helps users monitor refrigerator conditions remotely, identify potential refrigeration problems at an early stage, and maintain suitable storage conditions for food preservation.

3. PROPOSED SOLUTION

The proposed Smart Refrigerator Monitoring System uses a DHT11 temperature and humidity sensor and an ESP8266 NodeMCU to continuously monitor the environmental conditions inside a refrigerator. The DHT11 sensor measures the internal temperature and humidity, while the ESP8266 reads and processes the sensor data. The collected information is then transmitted to a cloud platform using Wi-Fi, allowing users to monitor refrigerator conditions remotely.

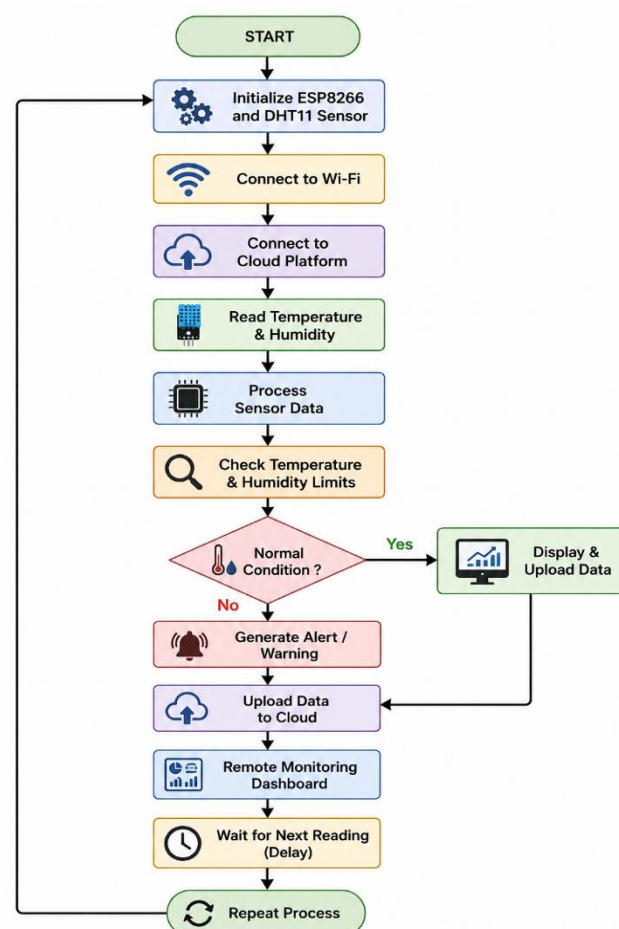
The system can identify abnormal conditions such as excessively high temperature or unusual humidity levels, which may indicate problems with the refrigeration system. The stored cloud data can be viewed through a dashboard and analyzed to observe environmental changes over time. This enables continuous monitoring and helps users maintain suitable conditions for food preservation.

The system operates as follows:

- **Temperature Monitoring:** Continuously measures the internal refrigerator temperature using the DHT11 sensor. The ESP8266 reads the temperature value at regular intervals.

- **Humidity Monitoring:** Measures the relative humidity inside the refrigerator and monitors changes in environmental conditions.
 - **Data Processing:** The ESP8266 receives and processes the sensor readings and determines the current refrigerator condition.
 - **Wireless Communication:** The ESP8266 uses Wi-Fi to transmit the collected data to the cloud platform.
 - **Cloud Data Storage:** Temperature and humidity readings are stored on the cloud platform for remote monitoring and future analysis

4. SYSTEM ARCHITECTURE



- Temperature and humidity sensor – measures the internal refrigerator conditions.
- ESP8266/ESP32 microcontroller – processes the sensor readings.
- Wi-Fi communication – transfers the collected data to the internet.
- Cloud platform – stores and visualizes temperature and humidity data.
- Monitoring dashboard – allows users to view real-time and historical readings.
- Alert system – provides warnings when temperature or humidity exceeds predefined limits.

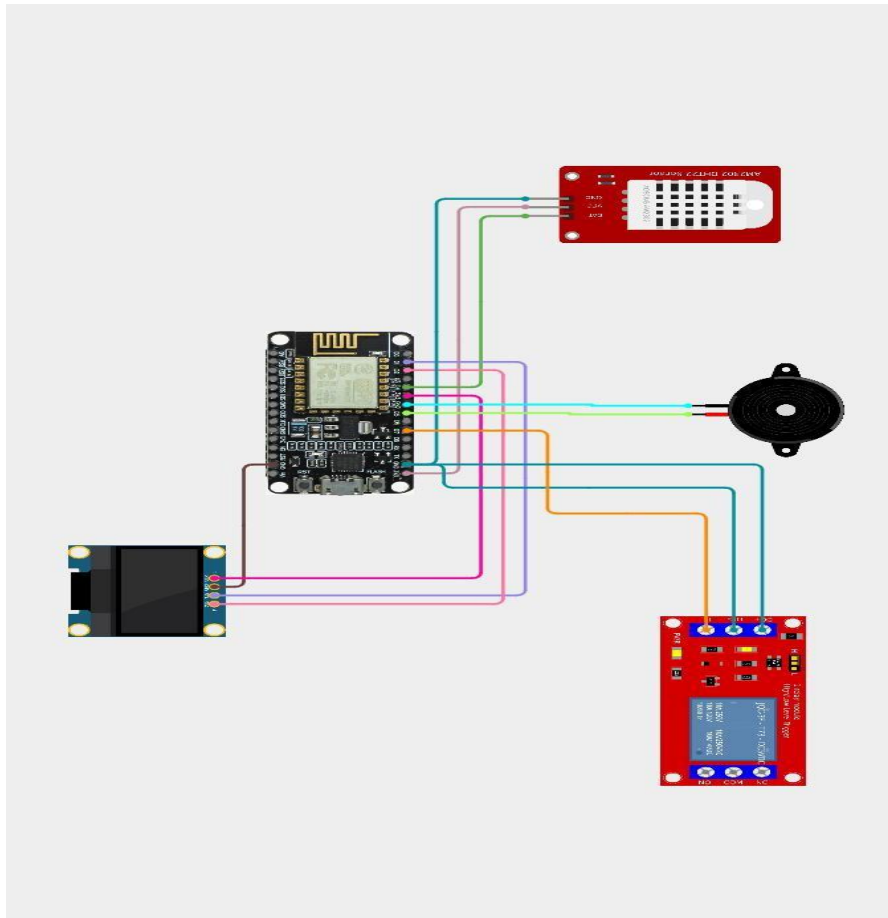
- Continuous monitoring – repeatedly collects and uploads environmental data for reliable refrigerator condition tracking.

5. CIRCUIT TABLE

Circuit Table

Component	Connection
DHT11 Temperature & Humidity Sensor	Data → D4 (GPIO2)
OLED/LCD Display	SDA → D2 (GPIO4), SCL → D1 (GPIO5)
Buzzer	Signal → D5 (GPIO14)
Relay Module	IN → D6 (GPIO12)
ESP8266 NodeMCU	3.3V & GND → All Components
Wi-Fi Network	Wireless Connection
Cloud Platform	Receives Data from ESP8266

CIRCUIT DESIGN



7. ALGORITHM

- Start the Smart Refrigerator Monitoring System.
- Initialize the temperature and humidity sensor.
- Initialize the ESP8266/ESP32 microcontroller.
- Establish a Wi-Fi connection.
- Read the current temperature and humidity values.
- Process and validate the sensor readings.
- Compare the readings with predefined safe limits.
- Display the temperature and humidity values.
- Upload the sensor data to the cloud platform through Wi-Fi.
- Store the readings for real-time and historical monitoring.
- If the values are within the safe range, continue normal monitoring.
- If the values exceed the safe limit, generate an alert.
- Repeat the measurement and data-upload process continuously.
- Stop the system when the power supply is switched OFF.
-

8. CODING

```
#include <WiFi.h> #include <Wire.h>
```

```
#include <Adafruit_MQTT.h> #include <Adafruit_MQTT_Client.h> #include <DHT.h>
```

```

#include <hd44780.h>

#include <hd44780ioClass/hd44780_I2Cexp.h>

// =====

// WIFI DETAILS

// =====

#define WIFI_SSID "OnePlus Nord CE5 7C5C" #define WIFI_PASSWORD "qkts5737"

// =====

// ADAFRUIT IO DETAILS

// =====

#define AIO_SERVER      "io.adafruit.com" #define AIO_SERVERPORT 1883
#define AIO_USERNAME "Sivasaravanan"
#define AIO_KEY  "aio_ADVS50c7oG5Sx1R3FskFAq0ŦUrB"

// =====

// DHT11

// =====

#define DHTPIN 15 #define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

// =====

// I2C LCD

// =====


hd44780_I2Cexp lcd; #define SDA_PIN 21
#define SCL_PIN 22

// =====

// RELAY

// =====

#define RELAY_PIN 26

// Most common relay modules are ACTIVE LOW #define RELAY_ON LOW
#define RELAY_OFF HIGH

// =====

```

```
// BUZZER

// =====

#define BUZZER_PIN 25

// =====

// MQTT

// =====

WiFiClient client;

Adafruit_MQTT_Client mqtt( &client,
AIO_SERVER, AIO_SERVERPORT, AIO_USERNAME, AIO_KEY
);

// =====

// ADAFRUIT IO FEEDS

// =====

// Temperature feed

Adafruit_MQTT_Publish temperatureFeed = Adafruit_MQTT_Publish(
&mqtt,
AIO_USERNAME "/feeds/hvac-temperature"
);

// Humidity feed

Adafruit_MQTT_Publish humidityFeed = Adafruit_MQTT_Publish(
&mqtt,
AIO_USERNAME "/feeds/hvac-humidity"
);

// Status feed

Adafruit_MQTT_Publish statusFeed = Adafruit_MQTT_Publish(
&mqtt,
AIO_USERNAME "/feeds/hvac-status"
);

// Buzzer feed
```



```
Adafruit_MQTT_Publish buzzerFeed = Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/hvac-buzzer"
);
// =====

// CONNECT TO WIFI
// =====

void connectWiFi()
{
    Serial.println();

    Serial.println("Connecting to WiFi...");

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    while (WiFi.status() != WL_CONNECTED)
    {
        delay(500);
        Serial.print(".");
    }
    Serial.println();
    Serial.println("WiFi Connected!"); Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
}
// =====

// CONNECT TO ADAFRUIT IO
// =====

bool connectMQTT()
```

```
{
if (mqtt.connected())
{
return true;
}
Serial.println();
Serial.println("Connecting to Adafruit IO..."); int8_t ret = mqtt.connect();
if (ret != 0)
{
Serial.print("MQTT Connection Failed: ");

Serial.println(mqtt.connectErrorString(ret)); mqtt.disconnect();
return false;
}
Serial.println("Adafruit IO Connected!"); return true;
}

// =====
// SETUP
// =====

void setup()
{
Serial.begin(115200);

//
// DHT11
// dht.begin();
//
// I2C LCD
//
Wire.begin(SDA_PIN, SCL_PIN); int lcdStatus = lcd.begin(16, 2); lcd.backlight();
lcd.clear();
```

```
lcd.setCursor(0, 0);  
lcd.print("HVAC MONITOR"); lcd.setCursor(0, 1);  
lcd.print("Starting...");  
//  
  
// RELAY  
//  
pinMode(RELAY_PIN, OUTPUT);  
digitalWrite(RELAY_PIN, RELAY_OFF);  
//  
// BUZZER  
//  
pinMode(BUZZER_PIN, OUTPUT);  
digitalWrite(BUZZER_PIN, LOW); delay(2000);  
//  
// WIFI  
//  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("WiFi Connecting"); connectWiFi();  
//  
// MQTT  
//  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("Adafruit IO"); lcd.setCursor(0, 1);  
lcd.print("Connecting..."); connectMQTT();  
delay(2000); lcd.clear();  
}
```

```
// =====  
// LOOP  
// =====  
  
void loop()  
{  
// =====  
// CHECK WIFI  
// =====  
  
if (WiFi.status() != WL_CONNECTED)  
{  
Serial.println("WiFi disconnected!"); connectWiFi();  
}  
// =====  
// CHECK MQTT  
// =====  
  
if (!connectMQTT())  
{  
Serial.println("MQTT not connected."); delay(5000);  
return;  
}  
// =====  
// READ DHT11  
// =====  
  
float temperature = dht.readTemperature(); float humidity = dht.readHumidity();  
// =====  
// SENSOR ERROR  
  
// =====  
  
if (isnan(temperature) || isnan(humidity))  
{
```

```

Serial.println("DHT11 ERROR!"); lcd.clear();

lcd.setCursor(0, 0);

lcd.print("DHT11 ERROR"); lcd.setCursor(0, 1);

lcd.print("Check Wiring");

digitalWrite(RELAY_PIN, RELAY_OFF); digitalWrite(BUZZER_PIN, LOW); delay(2000);

return;

}

// =====

// HVAC STATUS

// =====

String hvacStatus;

String buzzerStatus;

if (temperature > 30.0)

{

// High temperature

digitalWrite(RELAY_PIN, RELAY_ON); digitalWrite(BUZZER_PIN, HIGH); hvacStatus = "HIGH
TEMPERATURE"; buzzerStatus = "ON";

}

else

{

// Normal temperature

digitalWrite(RELAY_PIN, RELAY_OFF); digitalWrite(BUZZER_PIN, LOW);

hvacStatus = "NORMAL"; buzzerStatus = "OFF";

}

// =====

// LCD DISPLAY

// =====

lcd.clear();

lcd.setCursor(0, 0); lcd.print("Temp:");

lcd.print(temperature, 1); lcd.write(223);

```

```

lcd.print("C");
lcd.setCursor(0, 1); lcd.print("Hum:");
lcd.print(humidity, 1); lcd.print("%");

// =====

// SERIAL MONITOR

// =====

Serial.println();
Serial.println("=====");
Serial.print("Temperature : "); Serial.print(temperature, 1); Serial.println(" C");
Serial.print("Humidity      : "); Serial.print(humidity, 1);

Serial.println(" %");
Serial.print("HVAC Status : "); Serial.println(hvacStatus);
Serial.print("Relay   : "); if (temperature > 30.0)
Serial.println("ON"); else
Serial.println("OFF");
Serial.print("Buzzer : "); Serial.println(buzzerStatus);

// =====

// SEND TEMPERATURE

// =====

if (temperatureFeed.publish(temperature))
{
Serial.println("Temperature -> Adafruit IO : OK");
}
else
{
Serial.println("Temperature -> Adafruit IO : FAILED");
}

// =====

// SEND HUMIDITY

```

```
// =====  
if (humidityFeed.publish(humidity))  
{  
Serial.println("Humidity -> Adafruit IO : OK");  
}  
else  
  
{  
Serial.println("Humidity -> Adafruit IO : FAILED");  
}  
// =====  
// SEND HVAC STATUS  
// =====  
if (statusFeed.publish(hvacStatus.c_str()))  
{  
Serial.println("Status -> Adafruit IO : OK");  
}  
else  
  
{  
Serial.println("Status -> Adafruit IO : FAILED");  
}  
// =====  
// SEND BUZZER STATUS  
// =====  
if (buzzerFeed.publish(buzzerStatus.c_str()))  
{  
Serial.println("Buzzer -> Adafruit IO : OK");  
}  
else  
  
{
```

```

Serial.println("Buzzer -> Adafruit IO : FAILED");

}

Serial.println("=====");

// =====

// WAIT 5 SECONDS

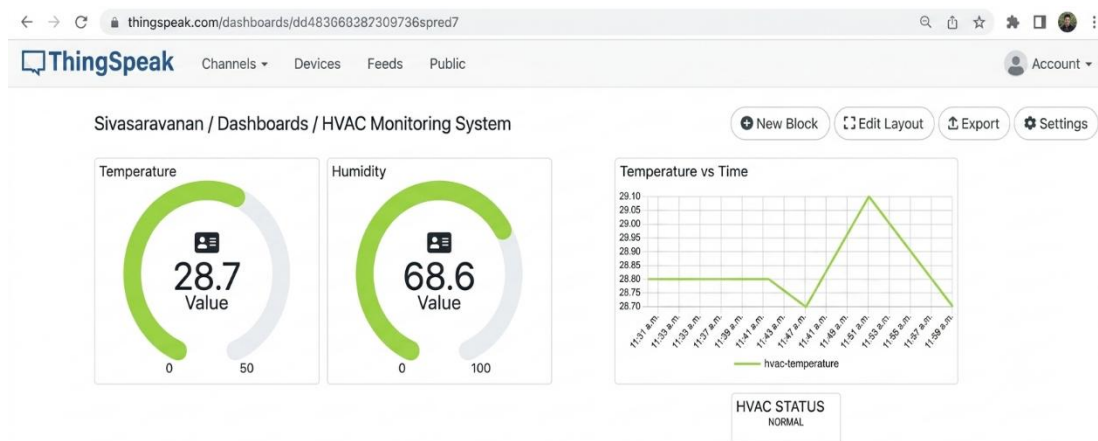
// =====

delay(5000);

}

```

9. WORKING SYSTEM



Working sequence:

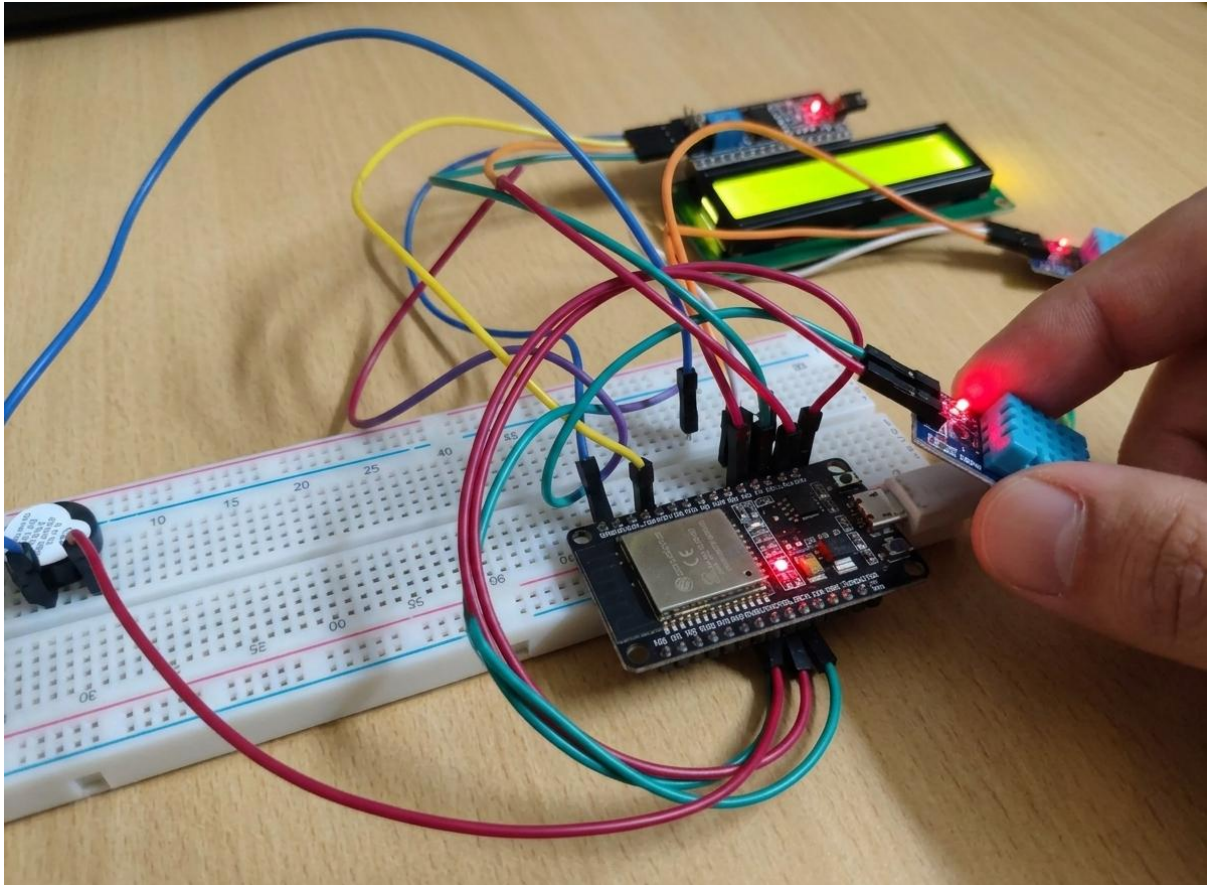
Start → Initialize ESP8266 & DHT11 Sensor → Connect to Wi-Fi → Connect to Cloud Platform → Read Temperature & Humidity → Process Sensor Data → Check Threshold Limits → Display

Values → Generate Alert (if required) → Upload Data to Cloud → Remote Monitoring Dashboard
→ Repeat Process

10. BIL OF MATERIALS

S.No.	Component	Quantity
1	ESP8266 NodeMCU	1
2	DHT11 Temperature & Humidity Sensor	1
3	OLED/LCD Display	1
4	Relay Module	1
5	Buzzer	1
6	Breadboard	1
7	Jumper Wires	1 Set
8	USB Cable / Power Supply	1

11.PROTOTYPE IMPLEMENTATION



12.RESULT & PERFORMANCE ANALYSIS

Performance:

The system provides:

- Continuous temperature monitoring inside the refrigerator.
- Continuous humidity monitoring.
- Real-time sensor data acquisition.
- Wi-Fi based wireless communication.
- Cloud-based data storage.
- Remote monitoring through a dashboard.
- Historical data visualization and analysis.
- Detection of abnormal temperature and humidity conditions.

Advantages

1. Continuous monitoring of refrigerator conditions.
2. Remote access through a cloud dashboard.
3. Early detection of refrigeration problems.
4. Cloud-based storage of sensor data.
5. Historical trend analysis of temperature and humidity.
6. Wireless communication using Wi-Fi.
7. Low-cost and easy-to-implement IoT solution.

Limitations

1. The system depends on Wi-Fi and Internet availability.
2. DHT11 provides limited accuracy compared to industrial-grade sensors.
3. Sensor performance may be affected by environmental conditions.
4. Cloud monitoring is unavailable during network failures.
5. The prototype mainly monitors conditions and does not directly control refrigerator cooling.
6. Power interruptions can temporarily stop data collection and transmission.